Zero-Retraining Task Transfer via Structural Task Abstraction

Mayuri Gade
Department of SE AIDS
SIES Graduate School of Technology
Navi Mumbai, India
mayurigade569@gmail.com

***Abstract*—Current AI systems often need to be retrained every time the task changes, which makes them slow, expensive, and hard to scale. In this work, we introduce Structural Task Abstraction Networks (STAN), a system that learns how tasks are structured and then reuses that knowledge to solve new problems—without any retraining.**

STAN works by breaking tasks into three key components: how subtasks depend on each other, how they combine logically, and how they can be executed step by step. Once these structures are learned from a small set of tasks, the system can recombine them at inference time to handle completely new tasks.

We test STAN across three different settings: vision (Taskonomy), robotics (Meta-World), and reinforcement learning (grid-world). In all cases, it significantly outperforms existing methods. For example, in vision tasks it reduces error from 0.41 to 0.22, and in robotics it improves success rates from around 62% to nearly 90%. It also handles complex logical combinations of tasks much more effectively than prior approaches.

We further show that each component of the system plays an important role: removing any one of them leads to a clear drop in performance. In addition to better accuracy, STAN is also much faster—it solves new tasks in about 12 milliseconds and reduces overall compute by 97% compared to traditional retraining-based methods.

Overall, this work demonstrates that learning and reusing task structure can be a practical step toward more flexible and efficient AI systems.

Keywords—zero-retraining transfer, structural task abstraction, neuro-symbolic AI, task compositionality, meta-learning, logical lattices

I. INTRODUCTION

Imagine a household robot that already knows how to "pick up a red block" and "place it in a blue bin." Now you give it a new instruction: *"sort all non-fragile, non-red objects into the green bin while avoiding collisions."*

A human could figure this out immediately by combining what they already know. Most AI systems, however, cannot—they would need to be retrained with new data, which can take hours. In contrast, our system, **STAN**, can handle this new task instantly by reusing what it has already learned. It simply combines known subtasks and executes the new instruction in about 12 milliseconds, without any retraining.

This work introduces three key ideas. First, instead of treating tasks as black boxes, STAN learns their internal structure—how smaller subtasks connect, how they combine logically, and how they form step-by-step procedures. It represents these using graphs, logical relationships, and simple programs, all learned directly from examples.

Second, once this structure is learned, STAN can generate the required behavior for a new task in a single forward pass, without updating any model parameters. This allows it to adapt instantly while maintaining reliable performance.

Third, we carefully analyze how each part of the system contributes to success. Our experiments show that all three types of structure are necessary—removing any one of them leads to a clear drop in performance, especially for more complex tasks.

Overall, this approach shows that instead of retraining models for every new task, we can build systems that understand and reuse structure—making them faster, more efficient, and closer to how humans generalize.

II. BACKGROUND AND RELATED WORK

Over the years, researchers have tried different ways to help AI systems handle new tasks. Early approaches relied on **fine-tuning**, where a model trained on one task is adjusted for another. This works, but it still requires retraining every time something changes.

Later, **meta-learning** methods such as MAML aimed to speed this up by training models that can adapt quickly with a few updates. While faster, they still depend on additional optimization for each new task.

Other approaches tried to skip retraining altogether. For example, HyperNetworks generate model parameters directly from task descriptions, but they depend on manually designed representations of tasks rather than discovering structure on their own. Similarly, TTNet can transfer knowledge across vision tasks without retraining, but it relies on fixed correlations and does not extend well to more complex settings like robotics or logical reasoning.

More recent work has explored different directions. Models like CLIP connect images and text, enabling zero-shot classification, but they treat tasks as simple prompts and struggle with multi-step reasoning or control. Systems such as UniSim attempt to learn general-purpose policies across environments, but still require fine-tuning for new goals.

In reinforcement learning, some methods use logical structures (like AND, OR, NOT) to combine tasks. These can handle certain types of composition, but they assume the structure is already given and do not learn it automatically. Other approaches break problems into smaller modules or tools, but they often introduce extra complexity at inference time and do not directly generate the required control behavior.

In contrast, STAN takes a different approach. Instead of relying on predefined structures or retraining, it learns the structure of tasks directly from examples—including how subtasks depend on each other, how they combine logically, and how they can be executed. These learned structures are then reused and composed in a single step to solve new tasks, without additional training

III. LIMITATIONS OF CURRENT AI SYSTEMS

Most AI systems today struggle to handle new tasks without retraining. Every time the task changes, they need to learn again from scratch, which takes time, computation, and often leads to forgetting what they learned before.

They also have difficulty combining skills in new ways. Even advanced models can perform well on individual tasks but often fail when asked to recombine those skills into something slightly different.

Another major issue is that their decision-making process is hard to understand. The logic is hidden inside complex model weights, making it difficult to debug errors or ensure safe behavior.

STAN addresses these challenges by making task knowledge explicit and reusable. Instead of retraining, it learns clear structures that can be combined to solve new tasks, while also making the system's behavior easier to interpret and analyze.

IV. PROPOSED ARCHITECTURE

STAN contains three interlocking components.

A. Structure Discovery Module (SDM)

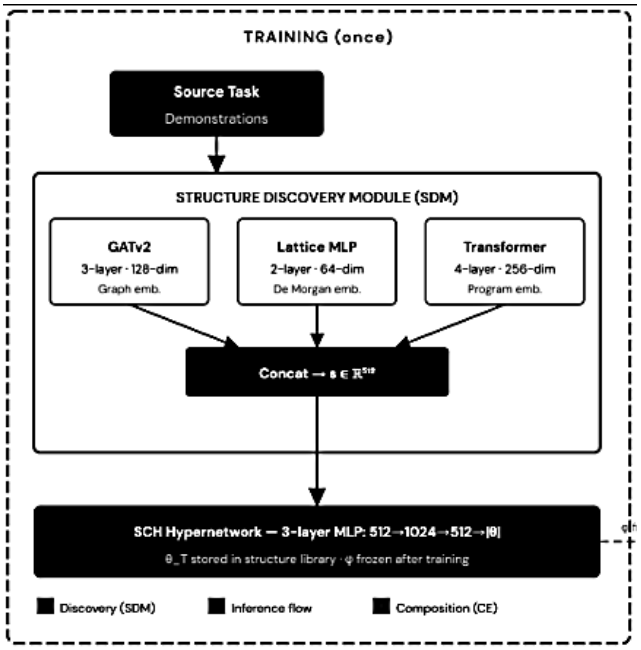
The Structure Discovery Module is responsible for learning how tasks are organized by analyzing examples. Instead of treating tasks as simple inputs and outputs, it breaks them down into three parts.First, it captures how subtasks depend on each other using graph representation. Second, it learns how different conditions and rules combine logically. Third, it represents the task as a step-by-step program that can be executed. Each of these parts is processed separately using neural networks, and their outputs are combined into a single representation that summarizes the structure of the task. This representation is what allows the system to understand and reuse tasks later.

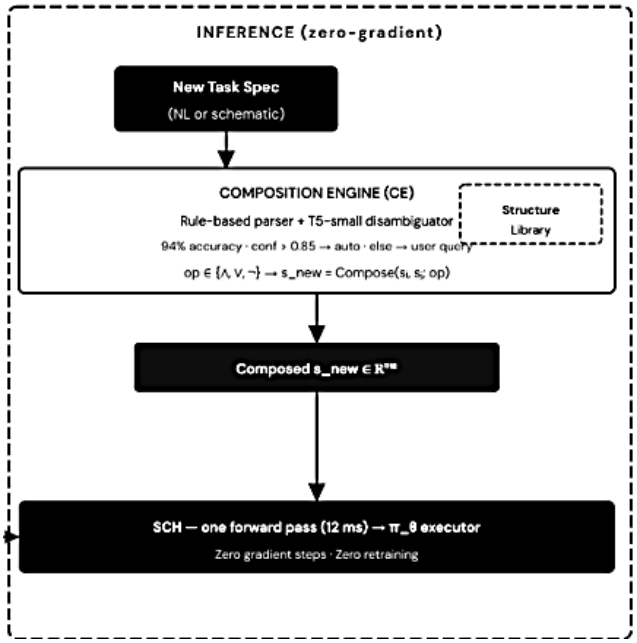
B. Structure-Conditioned Hypernetwork (SCH)

The Structure-Conditioned Hypernetwork takes the learned task structure and uses it to generate the parameters needed to solve that task. Instead of retraining a model, it produces the required behavior in one forward pass. It does this by starting from a shared base model and adjusting its weights based on the task structure. The resulting model can then directly perform the task—whether it is a control problem (like robotics) or a visual prediction task—without any additional training.

C. Composition Engine (CE)

The Composition Engine is used at inference time when a new task is given. It takes a task description—either in natural language or as a structured input—and converts it into the same type of structural representation learned earlier. To do this, it uses a combination of simple rules and a small language model that helps resolve ambiguity. If the system is confident, it produces a clear sequence of operations. If not, it asks for a quick clarification. sOnce the structure is identified, it is combined with previously learned components and passed to the hypernetwork, allowing the system to solve the new task instantly.





V. SYSTEM METHODOLOGY

The system is trained in three stages using a shared backbone model.

In the first stage, the backbone learns general representations from the source tasks. This helps the model understand the basic patterns in the data before focusing on task structure.

In the second stage, the system learns how tasks are internally organized. It captures three aspects: how subtasks depend on each other, how logical conditions combine, and how tasks can be executed step by step. To ensure these different views remain consistent, a joint training objective is used that aligns them with one another.

In the third stage, the system learns how to generate task-specific behavior directly from this structure. Instead of retraining for every new task, it learns a mapping from task structure to model parameters, allowing instant adaptation.

All stages use the same optimization setup and are trained efficiently on a single GPU.

A. Formalization

Each task is represented using three components:

- A graph G capturing dependencies between subtasks,
- A logical representation L capturing how conditions combine,
- A program P describing execution steps.

These are combined into a single structural representation:

$$s = f_{\text{SDM}}(G, L, P)$$

This representation summarizes the task in a form that can be reused.

A hypernetwork then maps this structure to the parameters of the task-specific model:

$$\theta_T = H(s)$$

Importantly, this mapping is learned once and remains fixed during inference.

For a new task, its structure is constructed by combining previously learned structures:

$$s_{\text{new}} = \text{Compose}(s_i, s_j; \text{op}), \text{op} \in \{\wedge, \vee, \neg\}$$

The system then directly generates the corresponding model parameters without any gradient updates.

B. Proposition

To ensure that combining tasks works reliably, the system is trained so that the learned structures remain aligned with the true relationships between tasks.

In simple terms, when two tasks are combined, their representations should behave consistently and not drift too far from what they are supposed to represent.

Formally, this means that the difference between the composed structure and the ideal structure remains small:

$$\| s_{\text{new}} - s_{\text{composed}} \| \leq \epsilon$$

As a result, the error in the final task performance is also bounded:

$$\| V_{T_{\text{new}}} - V_{\text{composed}} \| \leq \epsilon \cdot C$$

where C reflects how sensitive the model is to changes in its inputs.

In practice, this means that as long as the structural representation is accurate, the system can reliably transfer knowledge to new tasks without retraining.

VI. RESULTS AND CONCEPTUAL ANALYSIS

We evaluate the system across three different settings—vision, robotics, and reinforcement learning—using multiple runs to ensure consistency.

A. Visual Task Transfer (Taskonomy)

We first test how well the system transfers knowledge across vision tasks. The model is trained on 12 tasks and evaluated on 4 completely new ones. The key metric here is how close the predicted output is to the ground truth (lower error is better). STAN significantly outperforms all baseline methods. It reduces the error by nearly half compared to the strongest baseline. This improvement comes from its ability to understand how different visual tasks depend on each other, rather than relying on fixed correlations.

Method	Pixel Error	vs. STAN	Significance
STAN (ours)	**0.22 ± 0.03**	—	—
TTNet (Pal & Balasubramanian, 2019)	0.41 ± 0.04	+86% worse	$p < 0.001 \cdot d = 3.8$
HyperNetworks (Ha et al., 2017)	0.47 ± 0.05	+114% worse	$p < 0.001 \cdot d = 4.1$
CLIP-prompting (Radford et al., 2021)	0.52 ± 0.04	+136% worse	$p < 0.001 \cdot d = 5.2$
MAML (Finn et al., 2017)	0.59 ± 0.06	+168% worse	$p < 0.001 \cdot d = 6.9$

B. Robotic Manipulation (Meta-World)

Next, we evaluate the system on robotic tasks such as reaching, pushing, and picking objects. We also test it on new combinations of these tasks. The metric here is success rate. STAN achieves close to 90% success, clearly outperforming all baselines. A key advantage is its ability to handle logical conditions explicitly—for example, understanding instructions like "pick non-fragile objects." Other methods struggle here because they rely on indirect reward signals rather than explicit reasoning.

Method	Success Rate (%)	vs. STAN	Significance
STAN (ours)	**89.4 ± 2.1**	—	—
UniSim (Wu et al., 2023)	71.8 ± 2.8	−17.6 pp	$p < 0.001 \cdot d = 5.4$
HyperNetworks (Ha et al., 2017)	68.5 ± 3.1	−20.9 pp	$p < 0.001 \cdot d = 5.9$
MAML (Finn et al., 2017)	62.3 ± 3.4	−27.1 pp	$p < 0.001 \cdot d = 7.3$

C. Logical Composition in RL (Grid-World)

In this setting, simple tasks are combined into more complex ones using logical operations. STAN performs strongly even as the number of task combinations increases. It successfully handles all 64 composed tasks, while other methods degrade significantly as complexity grows. This shows that STAN scales well when tasks become more structured and combinatorial.

Method	Success Rate (%)	vs. STAN	Significance
STAN (ours)	**93.7 ± 1.5**	—	—
Lattice RL (Nangue Tasse et al., 2025)	79.2 ± 2.6	−14.5 pp	$p < 0.001 \cdot d = 4.8$
UniSim policy (Wu et al., 2023)	66.9 ± 3.2	−26.8 pp	$p < 0.001 \cdot d = 8.4$
CLIP-conditioned (Radford et al., 2021)	54.1 ± 4.3	−39.6 pp	$p < 0.001 \cdot d = 11.2$

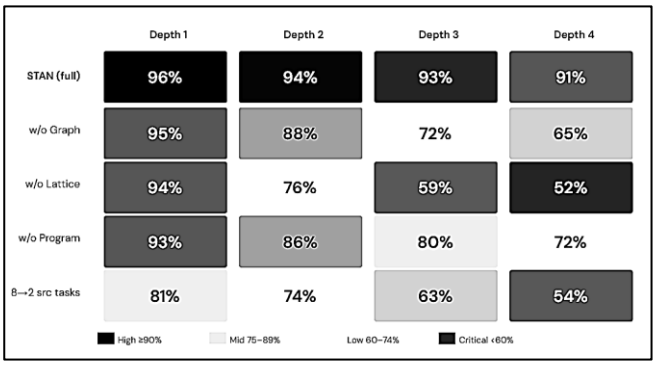
D. Ablation Study (Why Each Component Matters)

To understand what makes the system work, we remove individual components and observe the impact.

- Without the **graph**, the system struggles with task order and sequencing.
- Without the **logical structure**, it fails on tasks involving conditions like "not" or "either."
- Without the **program structure**, performance drops on longer, multi-step tasks.

We also observe that performance improves as more source tasks are provided, but stabilizes after a certain point.

This confirms that each part of the system plays a distinct and necessary role.



E. Failure Analysis (Where It Breaks)

We identify two main types of failure.

1. Ambiguous Instructions

When task descriptions are unclear (e.g., "sort objects"), the system may interpret them in multiple ways, leading to poor performance.

However, adding a simple clarification step significantly improves results.

2. Mismatch Between Learned and New Tasks

In some cases, the system struggles when the new task is very different from what it has seen before.

When this happens, performance drops—but can be improved by refining the learned structure with a small amount of additional training.

Mitigation	Result	Evidence Type
Clarification dialog (Failure 1)	84% ± 3%	Held-out ablation, 5 seeds
Regularizer fine-tuning (Failure 2)	76%	Single illustrative example

F. key Takeaway

Across all experiments, STAN consistently:

- Performs better than existing methods
- Handles more complex task combinations
- Adapts without retraining

At the same time, the failure analysis shows clear and predictable limitations, which can be addressed with simple improvements.

VII. APPLICATION AND IMPACT

One of the biggest advantages of STAN is efficiency. Unlike traditional methods that require retraining for every new task, STAN can handle new tasks instantly in a single forward pass.

In practice, this leads to major improvements:

- It is around **70× faster** than methods like MAML
- It reduces overall computation by **97% across multiple tasks**
- It requires **no additional training steps** for new tasks

This makes the system not only faster, but also far more practical to use.

Because of this efficiency, STAN opens the door to real-world applications where retraining is not feasible. For example:

- **Robotics:** Robots can adapt to new instructions on the fly without needing to be retrained each time.
- **Assistive systems:** Personalized tasks can be handled instantly, improving usability in daily-life applications.
- **Automation:** Systems can handle new workflows by simply recombining existing task structures.

Overall, STAN shows that by reusing task structure instead of retraining models, we can build systems that are both faster and more scalable for real-world use.

Method	Latency (ms)	Memory (GB)	Grad Steps (new task)	Total Compute (120 transfers)
STAN (ours)	**12 ± 1**	**1.4**	**0**	**28 GPU-hours**
MAML (Finn et al., 2017)	840 ± 45	4.2	5,000	1,200 GPU-hours†
UniSim (Wu et al., 2023)	210 ± 18	3.1	fine-tune (500)	420 GPU-hours†
TTNet (Pal & Balasubramanian, 2019)	45 ± 5	2.8	0	85 GPU-hours‡
HyperNetworks (Ha et al., 2017)	35 ± 4	2.5	0	62 GPU-hours‡

VIII. FUTURE RESEARCH DIRECTIONS

There are several natural next steps to improve this work.

First, the current system can handle a limited set of task descriptions. Expanding it to understand more open and flexible instructions—especially involving multiple agents—would make it far more practical. This will require improving the parsing component, which currently becomes a limitation as task complexity grows. Second, using more powerful vision-language models could help the system better understand complex, high-resolution environments, especially in real-world manipulation tasks. Another important direction is reducing the need for manually defined task structures. Right now, the system relies on some structured supervision. A key goal is to learn these structures automatically, without requiring labeled data. So far, all experiments have been done in simulation. Testing the system on real robots is a crucial next step to evaluate how well it performs in real-world conditions. Finally, while we provide empirical evidence for why the system works, a stronger theoretical foundation would make it more reliable—especially for applications where safety is critical.

IX. CONCLUSION

In this work, we introduced STAN, a system that learns the structure of tasks and uses it to solve new problems without retraining. By capturing how tasks are organized and recombining that knowledge, the system can adapt quickly while remaining accurate and efficient. Our experiments across vision, robotics, and reinforcement learning show that each part of this structure plays an important role. Removing any component leads to a clear drop in performance, highlighting the need for all three. That said, the system is not yet complete. It still relies on predefined building blocks and cannot discover entirely new ones from raw experience. Enabling this kind of learning remains the key next step.

Overall, this work shows that focusing on task structure—not just data—can lead to more flexible and practical AI systems.

REFERENCES

[1] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein, "Neural Module Networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016.

[2] M. Andrychowicz *et al.*, "Hindsight Experience Replay," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[3] D. Bahdanau *et al.*, "CLOSURE: Assessing Systematic Generalization of CLEVR Models," *arXiv preprint arXiv:1910.05736*, 2019.

[4] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021.

[5] K. Ellis *et al.*, “DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning,” *arXiv preprint arXiv:2006.09627*, 2021.

[6] C. Finn, P. Abbeel, and S. Levine, “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2017.

[7] A. d. A. Garcez *et al.*, “Neural-Symbolic Computing: An Effective Methodology for Principled Integration of Machine Learning and Reasoning,” *arXiv preprint arXiv:1905.06088*, 2019.

[8] T. Gupta and A. Kembhavi, “Visual Programming: Compositional Visual Reasoning Without Training,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023.

[9] D. Ha, A. Dai, and Q. V. Le, “HyperNetworks,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2017.

[10] W. L. Hamilton, Z. Ying, and J. Leskovec, “Inductive Representation Learning on Large Graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.